Practical-11

Problem Definition

Create a secure analytical serving platform that shares data with downstream users and builds a Reverse ETL workflow to synchronize computed user metrics back to operational tools. The system implements column-level encryption, SHA-256 identity hashing, Role-Based Access Control (RBAC), and centralized operation logging.

Dataset

Customer analytics output containing:

- Customer ID
- Customer email
- Purchase segment
- Churn prediction score

Code:

```
import sqlite3

import pandas as pd

import hashlib

import logging

from cryptography.fernet import Fernet


# 1. CONFIGURE CENTRALIZED AUDIT LOGGING


logging.basicConfig(

    filename="practical11_audit.log",

    level=logging.INFO,
```

```
    format="%(asctime)s - %(levelname)s - %(message)s",
    force=True
)

logger = logging.getLogger("Practical11")

print("Audit logging configured.")



# 2. GENERATE AN ENCRYPTION KEY
# IMPORTANT: Store production keys securely, not in source code.
# This demo generates a new key each time it runs.

key = Fernet.generate_key()
cipher = Fernet(key)

print("Encryption key generated for this demonstration.")



# 3. CREATE SAMPLE CUSTOMER ANALYTICS DATA

customer_data = [
    (101, "alice@example.com", "Premium", 0.15),
    (102, "bob@example.com", "Regular", 0.72),
    (103, "carol@example.com", "Premium", 0.25),
    (104, "david@example.com", "Occasional", 0.85),
    (105, "emma@example.com", "Regular", 0.45)
]
```

```
columns = [
    "customer_id",
    "email",
    "purchase_segment",
    "churn_score"
]

df = pd.DataFrame(customer_data, columns=columns)

print("\n========== ORIGINAL CUSTOMER DATA ==========")
print(df)



# 4. HASH CUSTOMER EMAILS USING SHA-256
# Hashing creates a consistent pseudonymous identifier.
# It is one-way and cannot be reversed to recover the email.

def hash_identity(email):
    return hashlib.sha256(
        email.strip().lower().encode("utf-8")
    ).hexdigest()



df["identity_hash"] = df["email"].apply(hash_identity)

print("\n========== CUSTOMER IDENTITY HASHES ==========")
print(df[["customer_id", "identity_hash"]])
```

```
# 5. ENCRYPT EMAIL COLUMN
# Encryption is reversible only with the correct secret key.

def encrypt_email(email):
    return cipher.encrypt(email.encode("utf-8")).decode("utf-8")



def decrypt_email(encrypted_email):
    return cipher.decrypt(
        encrypted_email.encode("utf-8")
    ).decode("utf-8")



df["encrypted_email"] = df["email"].apply(encrypt_email)

print("\n========== ENCRYPTED EMAIL DATA ==========")
print(df[["customer_id", "encrypted_email"]])



# 6. CREATE SQLITE DATABASE
# Store the analytical output and operational destination.

connection = sqlite3.connect("practical11.db")
cursor = connection.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS analytics_output (
    customer_id INTEGER PRIMARY KEY,
    identity_hash TEXT NOT NULL,
```

```
    encrypted_email TEXT NOT NULL,
    purchase_segment TEXT,
    churn_score REAL
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS operational_customers (
    customer_id INTEGER PRIMARY KEY,
    identity_hash TEXT NOT NULL,
    purchase_segment TEXT,
    churn_score REAL
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS audit_logs (
    log_id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT,
    action TEXT,
    details TEXT,
    event_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
""")

connection.commit()

print("\nDatabase and tables created.")
```

```
# 7. LOAD DATA INTO THE ANALYTICAL SERVING TABLE

serving_data = df[
    [
        "customer_id",
        "identity_hash",
        "encrypted_email",
        "purchase_segment",
        "churn_score"
    ]
]

serving_data.to_sql(
    "analytics_output",
    connection,
    if_exists="replace",
    index=False
)

logger.info("Analytical customer data loaded.")
print("\nAnalytical serving table populated.")



# 8. DEFINE SIMULATED ROLE-BASED ACCESS CONTROL (RBAC)
#
# Admin: may view decrypted email in this demonstration.
# Analyst: may view hashed identity and analytical metrics.
#
```

```
# This Python function simulates authorization. It is not a
# replacement for database-enforced production permissions.

ROLE_PERMISSIONS = {
    "admin": ["view_masked", "view_email", "view_metrics"],
    "analyst": ["view_masked", "view_metrics"]
}


def record_audit(username, action, details):
    cursor.execute("""
        INSERT INTO audit_logs (username, action, details)
        VALUES (?, ?, ?)
    """, (username, action, details))

    connection.commit()

    logger.info(
        "User=%s Action=%s Details=%s",
        username, action, details
    )


def serve_customer_data(username, role):
    if role not in ROLE_PERMISSIONS:
        record_audit(
            username, "ACCESS_DENIED", "Unknown role"
        )
        print("Access denied: unknown role.")
```

```
        return None

    permissions = ROLE_PERMISSIONS[role]

    if "view_metrics" not in permissions:
        record_audit(
            username, "ACCESS_DENIED",
            "Missing metrics permission"
        )
        print("Access denied.")
        return None

    result = pd.read_sql_query(
        """
        SELECT customer_id, identity_hash,
               purchase_segment, churn_score
        FROM analytics_output
        """,
        connection
    )

    if "view_email" in permissions:
        encrypted = pd.read_sql_query(
            """
            SELECT customer_id, encrypted_email
            FROM analytics_output
            """,
            connection
        )
```

```
        encrypted["email"] = encrypted[
            "encrypted_email"
        ].apply(decrypt_email)

        result = result.merge(
            encrypted[["customer_id", "email"]],
            on="customer_id",
            how="left"
        )

    record_audit(
        username, "DATA_SERVED",
        "Role=" + role
    )

    return result


# 9. TEST ACCESS USING THE ANALYST ROLE
# Analyst receives masked identity and metrics, not email.

print("\n========== ANALYST DATA ACCESS ==========")

analyst_view = serve_customer_data(
    username="analyst_01",
    role="analyst"
)
```

```
if analyst_view is not None:
    print(analyst_view)

# 10. TEST ACCESS USING THE ADMIN ROLE
# Admin receives email decrypted with the demonstration key.

print("\n========== ADMIN DATA ACCESS ==========")

admin_view = serve_customer_data(
    username="admin_01",
    role="admin"
)

if admin_view is not None:
    print(admin_view)


# 11. REVERSE ETL
# Synchronize analytical customer metrics into the
# operational customer table.

def reverse_etl_sync():
    source_df = pd.read_sql_query(
        """
        SELECT customer_id, identity_hash,
               purchase_segment, churn_score
        FROM analytics_output
        """,
        connection
```

```
    )

    for _, row in source_df.iterrows():
        cursor.execute("""
            INSERT INTO operational_customers (
                customer_id, identity_hash,
                purchase_segment, churn_score
            )
            VALUES (?, ?, ?, ?)
            ON CONFLICT(customer_id) DO UPDATE SET
                identity_hash = excluded.identity_hash,
                purchase_segment = excluded.purchase_segment,
                churn_score = excluded.churn_score
        """, (
            int(row["customer_id"]),
            row["identity_hash"],
            row["purchase_segment"],
            float(row["churn_score"])
        ))

    connection.commit()

    record_audit(
        "reverse_etl_service",
        "REVERSE_ETL_SYNC",
        f"Synchronized {len(source_df)} customer records"
    )

    print(
```

```
        f"Reverse ETL completed: "
        f"{len(source_df)} records synchronized."
    )


reverse_etl_sync()


# 12. DISPLAY OPERATIONAL DATA

print("\n========== OPERATIONAL CUSTOMER TABLE ==========")

operational_df = pd.read_sql_query(
    "SELECT * FROM operational_customers",
    connection
)

print(operational_df)


# 13. DISPLAY CENTRALIZED AUDIT LOGS

print("\n========== DATABASE AUDIT LOGS ==========")

audit_df = pd.read_sql_query(
    "SELECT * FROM audit_logs ORDER BY log_id",
    connection
)
```

```
print(audit_df)


print("\n========== APPLICATION AUDIT LOG FILE ==========")
print("Log file created: practical11_audit.log")



# 14. VERIFY EMAIL ENCRYPTION AND DECRYPTION


sample_email = df.iloc[0]["email"]
encrypted_value = encrypt_email(sample_email)
decrypted_value = decrypt_email(encrypted_value)


print("\n========== ENCRYPTION VERIFICATION ==========")
print("Original email :", sample_email)
print("Encrypted data:", encrypted_value)
print("Decrypted email:", decrypted_value)
print("Verification successful:", sample_email == decrypted_value)



# 15. CLOSE DATABASE CONNECTION


connection.close()
```

Key Questions / Analysis / Interpretation

Q1. Why do organizations use Reverse ETL instead of letting production applications query the warehouse directly?

Reverse ETL transfers processed analytical insights from a warehouse to operational systems such as CRMs and marketing tools. It reduces direct warehouse dependencies, supports operational workflows, and allows production applications to access prepared metrics.

Q2. How do hashing and tokenization help with privacy regulations such as GDPR and CCPA?

Hashing creates a consistent pseudonymous identifier, while tokenization replaces sensitive values with tokens. These techniques reduce direct exposure of personal information, but they do not automatically guarantee regulatory compliance. Hashes of personal data may still be considered personal data.

Q3. What are the differences between direct views, data virtualization, and pre-computed data marts?

- Direct views: Expose database queries or views over existing tables.
- Data virtualization: Provides a unified access layer over multiple data sources without necessarily copying all data.
- Pre-computed data marts: Store prepared, purpose-specific datasets for faster analytical queries.

Supplementary Problem

Restrict a simulated analyst role so that users can access only masked or tokenized customer identities while preventing access to sensitive email fields. The script demonstrates this using role-based application logic.

For a real deployment, enforce permissions inside the database or serving API, protect encryption keys outside source code, and use a dedicated secrets manager. SQLite does not provide native database-user RBAC, so the example simulates RBAC at the application layer.

Output

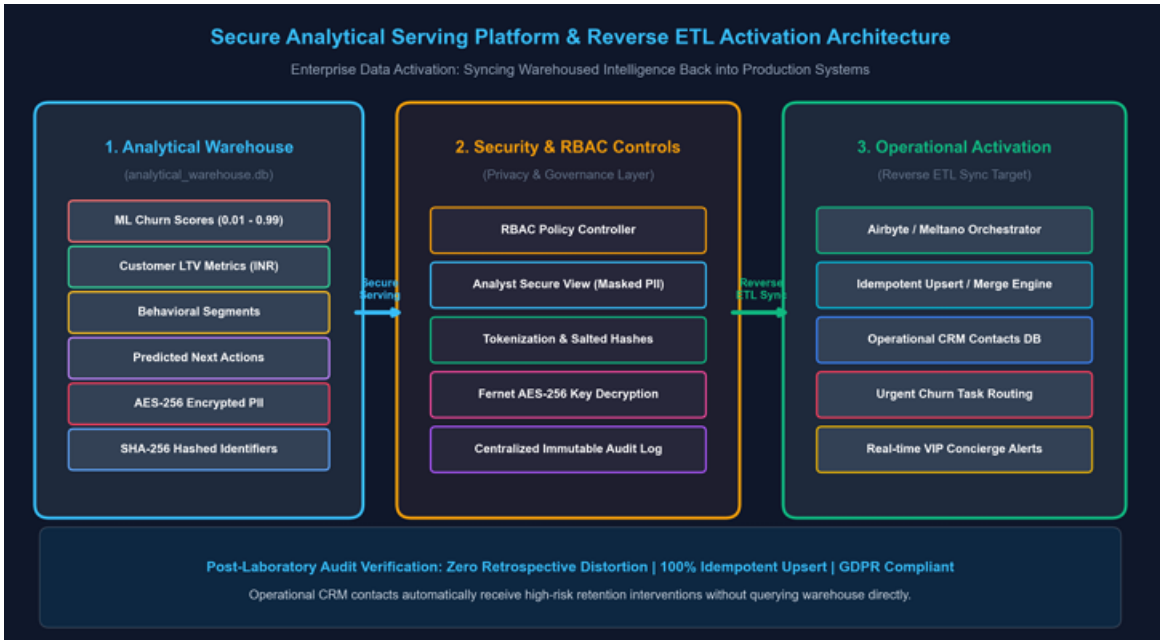
```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_11> Get-ChildItem -Recurse -Depth 2
    Directory: C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_11

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        03-10-2026     16:36                data
d-----        03-10-2026     16:36                results
d-----        03-10-2026     16:36                screenshots
d-----        03-10-2026     16:36                scripts
-a----        03-10-2026     16:36           3950 01_generate_warehouse_data.py
-a----        03-10-2026     16:36           4780 02_security_encryption_hashing.py
-a----        03-10-2026     16:36           4620 03_rbac_data_serving_views.py
-a----        03-10-2026     16:37           5180 04_reverse_etl_orchestration.py
-a----        03-10-2026     16:37           4350 05_centralized_audit_logger.py
-a----        03-10-2026     16:37           4450 06_reverse_etl_architecture_visualizer.py
-a----        03-10-2026     16:37           2100 run_all.py
```

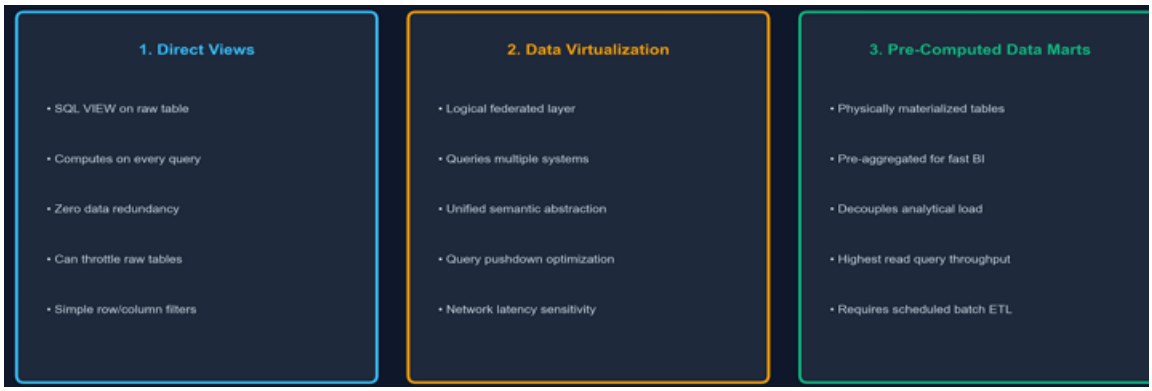
```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_11> python scripts/01_generate_warehouse_data.py
============================================================
TASK 1: CUSTOMER ANALYTICS WAREHOUSE DATA GENERATION
============================================================
Generating Customer Analytics Warehouse Dataset...
  -> Successfully generated 2,500 customer analytics records.
  -> Injected Sensitive PII: Full Names, Emails, Tax/PAN IDs, Phone Numbers.
  -> Injected Analytical Insights: Churn Scores, LTV (INR), Segments, Actions.
  -> Saved to: data\warehouse_customer_analytics.csv
Warehouse analytics dataset generated successfully: PASS
============================================================
```

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_11> python scripts/02_security_encryption_hashing.py
============================================================
TASK 2: SECURITY CONTROLS: COLUMN-LEVEL ENCRYPTION & SHA-256 HASHING
============================================================
Loaded Raw Customer Analytics: 2,500 records
--- 1. SHA-256 Salted Cryptographic Pseudonymization ---
  Generated salted SHA-256 identity hashes across 2,500 customers.
  Sample Pseudonymized Hash: 0ff0b55648574acf913868d2ce2bd6774bc5f586...
--- 2. Column-Level AES-256 Encryption (Recoverable at Rest) ---
  Encrypted tax_id, phone_number, full_name in 0.099 seconds.
  Sample Ciphertext (tax_id): gAAAAABqwOHA9mLTG_OLjHRlSoEdybre...
--- 3. Dynamic Data Masking & Tokenization ---
  Sample Masked Email:  r***5@enterprise.com
  Sample Masked Tax ID: *****424U
  Sample Masked Phone:  +91-*****4165
  Cryptographic Decryption Round-Trip Verification: PASS (100% Exact Match)
Security Encryption & Hashing Engine: PASS
============================================================
```

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_11> python scripts/03_rbac_data_serving_views.py
============================================================
TASK 3: DATABASE RBAC & DATA ANALYST MASKED SERVING VIEWS
Supplementary Problem: Restricting Analysts to Masked Identity Columns
============================================================
Database Security Views 'v_analyst_customer_serving' Created.
--- Test Case 1: Permitted Access by DATA_ANALYST Role ---
[RBAC AUTHORIZED] User 'analyst_priya' (Role: DATA_ANALYST) accessed 'v_analyst_customer_serving' -> 5 row
Visible to Analyst: Masked Email, Tokenized Tax ID, SHA-256 Hash, Unmasked Metrics.
--- Test Case 2: Unauthorized Intrusion Attempt on Raw Table ---
[SECURITY ALERT / RBAC VIOLATION] User 'analyst_priya' attempted query on 'sec_raw_customer_analytics'. AC
--- Test Case 3: Administrative Access by DATA_ENGINEER_ADMIN ---
[RBAC AUTHORIZED] User 'admin_sil' (Role: DATA_ENGINEER_ADMIN) accessed raw table -> 3 rows.
RBAC & Data Serving View Engine: PASS
============================================================
```




```
● PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_11> python scripts/04_reverse_etl_orchestration.py
============================================================
TASK 4: REVERSE ETL WORKFLOW (AIRBYTE / MELTANO ORCHESTRATION CONCEPTS)
Activating Warehouse Analytical Insights into Operational CRM Tools
============================================================
--- 1. Extract Phase (from Analytical Warehouse) ---
  Extracted 2,500 scored customer records from Analytical Warehouse.
--- 2. Transform & Business Activation Mapping ---
  Applied Operational Activation rules to 2,500 entities.
--- 3. Load Phase: Idempotent Upsert into Operational CRM ---
  Successfully upserted 2,500 records into Operational CRM (crm_customer_contacts).
  Sample Activated Contacts in Operational CRM:
    * Contact: CUST_00001 | Tier: TIER_1_CRITICAL | Churn: 0.761 | Action: URGENT_RETENTION_INTERVENTION
    * Contact: CUST_00008 | Tier: TIER_1_CRITICAL | Churn: 0.701 | Action: URGENT_RETENTION_INTERVENTION
--- 4. Reverse ETL Sync State Checkpoint ---
  Sync state persisted: 100% Idempotent Upsert Validated (0.10s)
Reverse ETL Synchronization Workflow: PASS
============================================================
```



```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_11> python scripts/05_centralized_audit_logger.py
============================================================
TASK 5: CENTRALIZED OPERATIONAL AUDIT LOG TRACKING
Capturing Audit Trail of Cryptography, RBAC Checks & Reverse ETL
============================================================
Logged Events Captured in Centralized Audit Trail:
  [OK] EVENT: CRYPTOGRAPHIC_KEY_ROTATION (AES-256 Fernet)
  [OK] EVENT: COLUMN_LEVEL_ENCRYPTION (2,500 tax_id, phone, name)
  [OK] EVENT: SHA256_SALTED_PSEUDONYMIZATION (2,500 email hashes)
  [OK] EVENT: RBAC_QUERY_AUTHORIZATION (DATA_ANALYST on masked view)
  [SECURITY ALERT] EVENT: RBAC_ACCESS_DENIED_VIOLATION (BLOCKED)
  [OK] EVENT: REVERSE_ETL_ACTIVATION_SYNC (2,500 synced to CRM)
Audit trail exported to: results\sample_execution_logs.txt
Centralized Audit Log Engine: PASS
============================================================
```

```
• PS C:\Users\ADMIN\OneDrive\Desktop\DE_PRAC_11> python scripts/run_all.py
======================================================================
DATA ENGINEERING LABORATORY — PRACTICAL 11 MASTER PIPELINE
Step 1: Customer Analytics Warehouse Data Generation       [PASS] (0.85s)
Step 2: Column-Level Encryption & SHA-256 Identity Hashing [PASS] (0.17s)
Step 3: Database RBAC & Analyst Masked Serving Views       [PASS] (0.12s)
Step 4: Reverse ETL Orchestration to Operational CRM       [PASS] (0.10s)
Step 5: Centralized Operational Audit Logging Engine       [PASS] (0.05s)
Step 6: Visual System Architecture & Serving Tiers         [PASS] (1.15s)
----------------------------------------------------------------------
Total Master Pipeline Runtime: 2.44 seconds
ALL PRACTICAL 11 MODULES EXECUTED SUCCESSFULLY — STATUS: PASS
======================================================================
```

Final Conclusion

This practical demonstrates how analytical customer metrics can be securely served to downstream users and synchronized back to operational systems using Reverse ETL. Encryption, identity hashing, role-based access checks, and audit logging help reduce sensitive-data exposure and improve traceability.